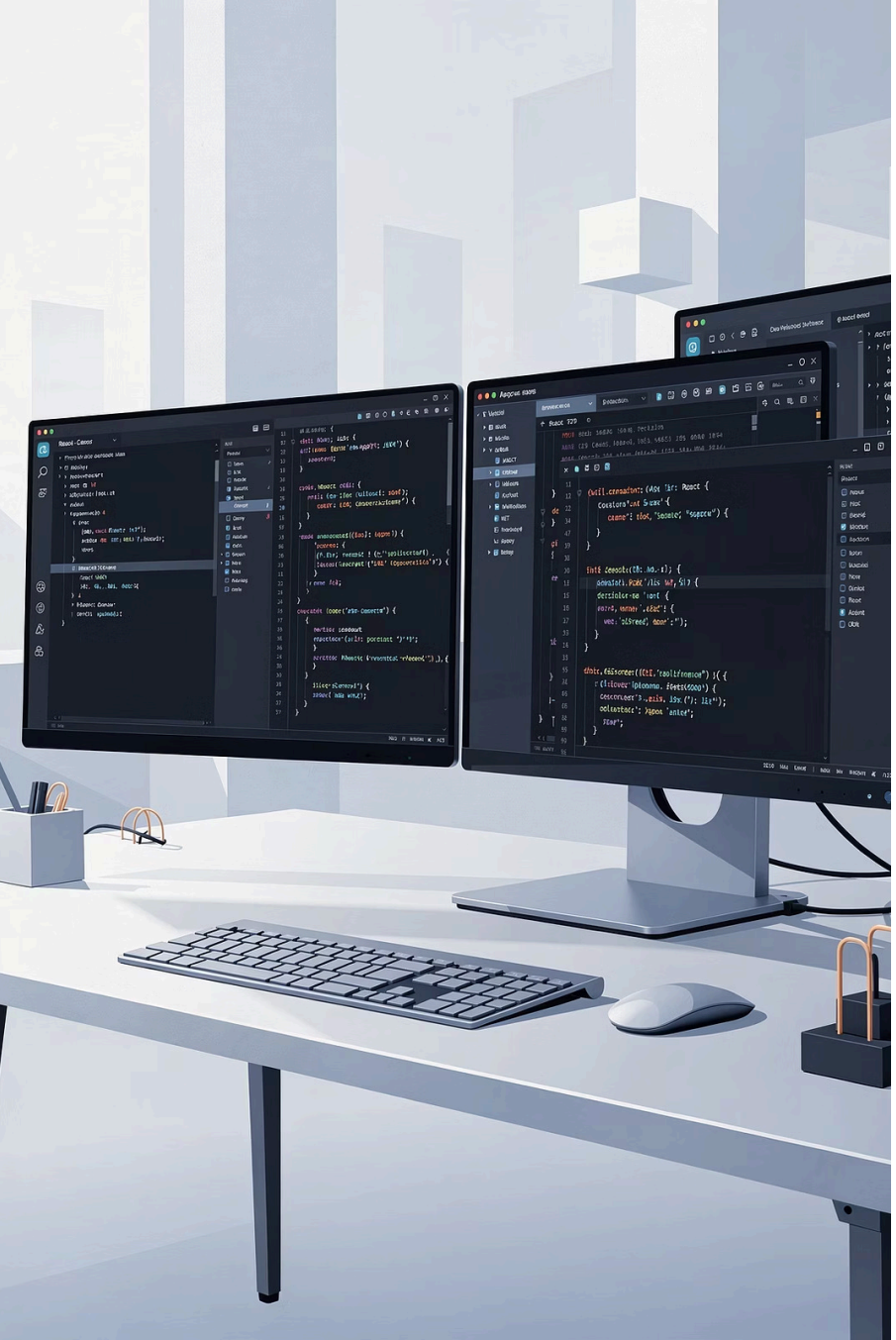


Desarrollo de Sitio Web con React y ASP.NET Core

Estudiante

Felix Hernandez · Matrícula: 20-0143

Asignatura

Programación Web II

Docente

Johanna Rodriguez R.

Introducción

El desarrollo web moderno combina tecnologías especializadas para construir aplicaciones robustas y dinámicas. **React** lidera en la creación de interfaces interactivas, mientras que **ASP.NET Core** proporciona la base para APIs seguras y escalables.

¿Qué se desarrolló?

Un sitio web completo que integra componentes reutilizables, manejo de estado, navegación entre páginas y consumo de datos desde una API.

¿Por qué es relevante?

Refleja el flujo real de trabajo utilizado en la industria del desarrollo web full-stack actual.

Objetivo del Proyecto

Crear un sitio web funcional con React que demuestre los conceptos clave aprendidos en la asignatura **Programación Web II**.

1

Interfaz Completa

Landing page y múltiples páginas navegables

2

Componentes

Diseño basado en componentes reutilizables

3

Estado

Manejo dinámico del estado de la aplicación

4

API REST

Conexión con backend en ASP.NET Core

Tecnologías Utilizadas

Frontend

React · JavaScript · JSX · CSS

Backend

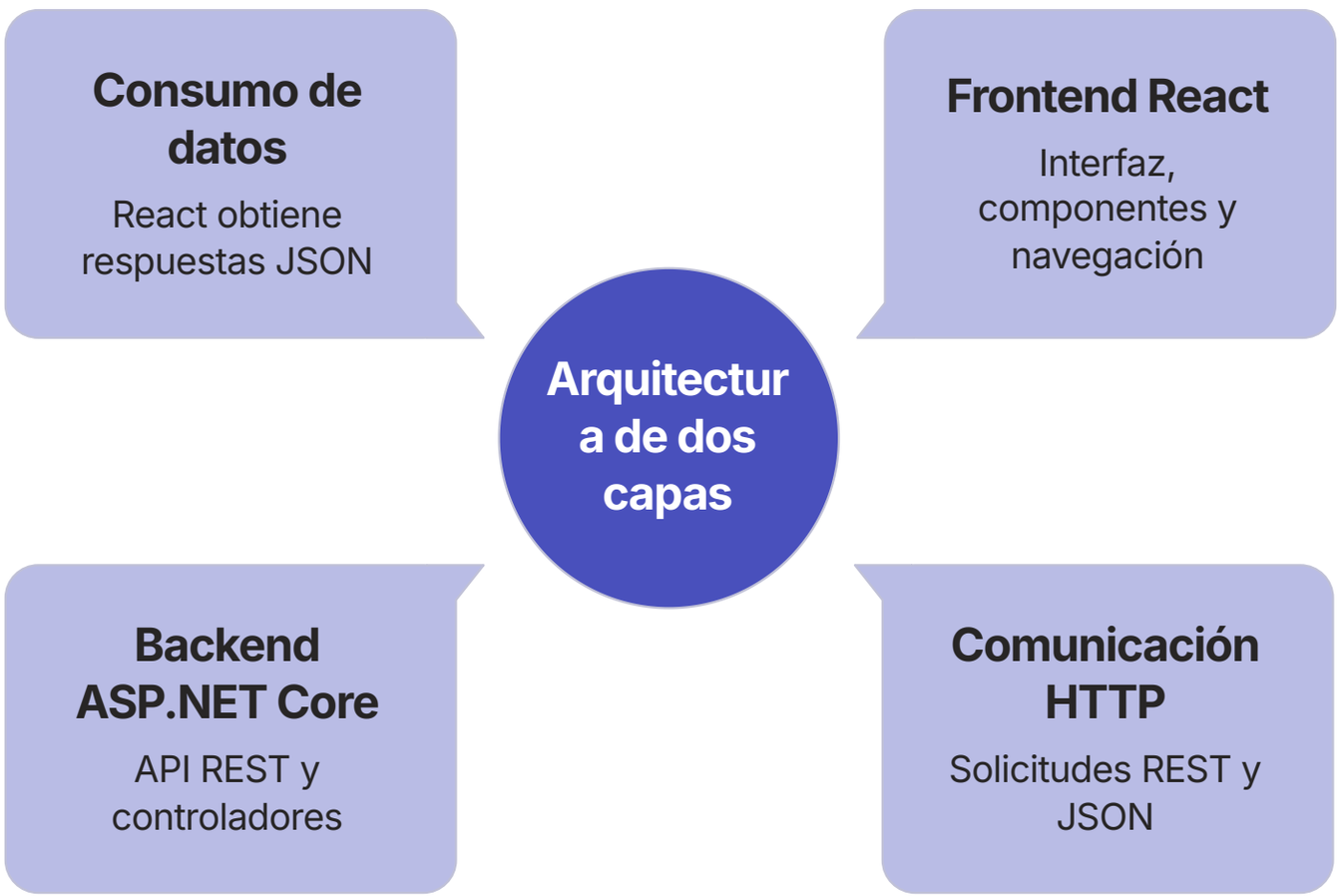
ASP.NET Core Web API · C#

Herramientas

Visual Studio · VS Code · GitHub

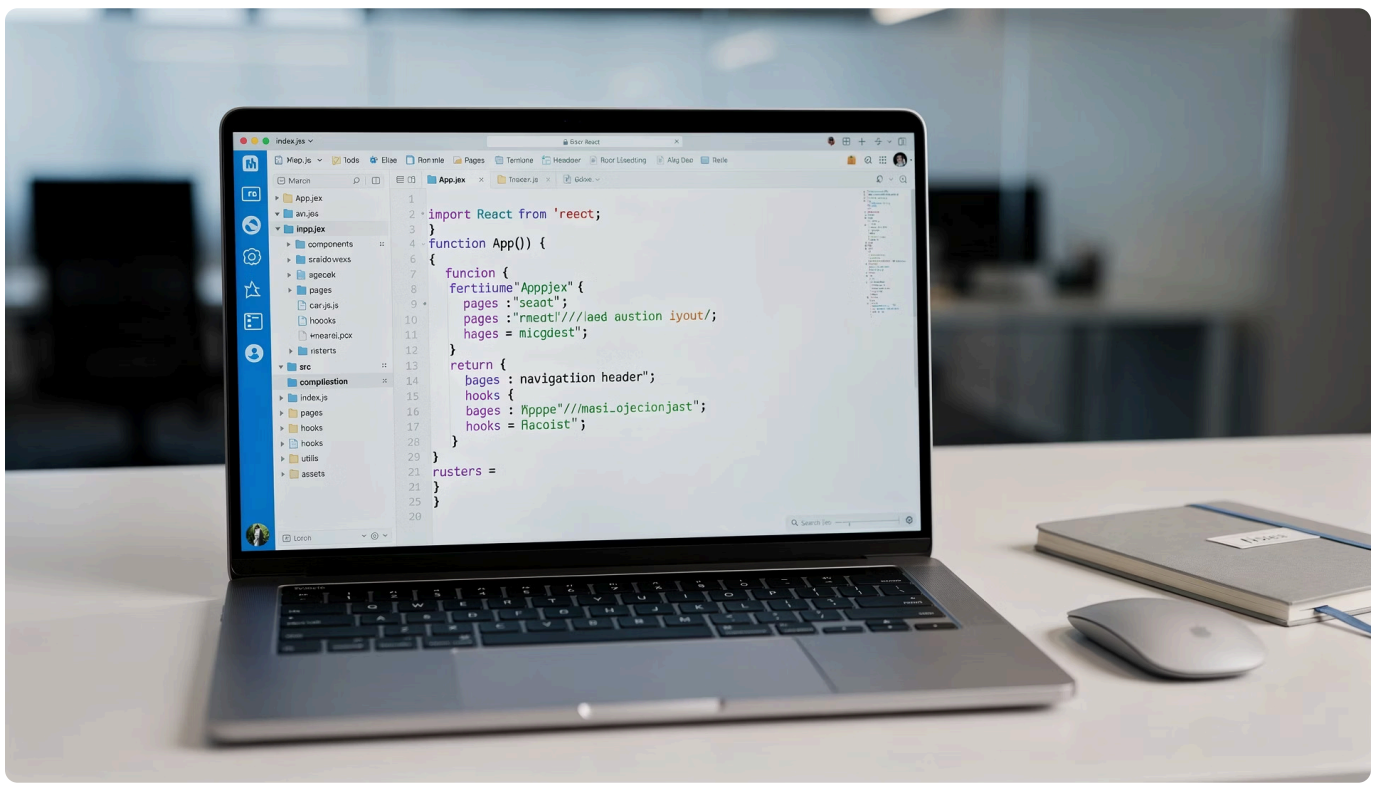
Arquitectura del Proyecto

El proyecto adopta una arquitectura **cliente-servidor desacoplada**. El frontend y el backend operan de forma independiente y se comunican mediante solicitudes HTTP.



React consume los datos del backend a través de peticiones HTTP, recibiendo respuestas en formato **JSON** listo para renderizar en la interfaz.

Estructura del Proyecto React



→ **components/**

Header, Footer, Navbar, Counter y más componentes reutilizables

→ **pages/**

Home, About y Products — páginas principales del sitio

→ **services/**

Lógica de conexión con la API externa

→ **styles/ · assets/**

Estilos globales e imágenes del proyecto

Landing Page

La página principal del proyecto es **Home.jsx**, diseñada como landing page para recibir al usuario con información clara y componentes interactivos.

Presentación

Información general del proyecto y sus objetivos

Interactividad

Componentes dinámicos que demuestran el uso de React

Navegación

Acceso directo a las demás páginas del sitio

Navegación del Sitio

El sitio utiliza **React Router** para gestionar las rutas de la aplicación de forma declarativa, sin recargar la página. El componente **Navbar.jsx** provee acceso fluido entre secciones.

> React Router permite una experiencia de **Single Page Application (SPA)**, donde la navegación es instantánea y sin interrupciones.

Inicio

Landing page principal del proyecto

Nosotros

Información sobre el proyecto y el autor

Productos

Listado obtenido desde la API

Componentes del Proyecto

La aplicación se construye con **componentes reutilizables**, cada uno con una responsabilidad específica. Esto mejora la organización, el mantenimiento y la escalabilidad del código.

Estructura

Header · Footer · Navbar

Contenido

ProductCard

Interactivos

Counter · VisibilityToggle · ThemeStatus

Manejo de Estado con *useState*

El proyecto utiliza el Hook **useState** de React para gestionar datos dinámicos. Cada componente reacciona automáticamente a los cambios de estado, actualizando la vista en tiempo real.

Counter.jsx

Botones para incrementar o disminuir un contador numérico en pantalla.

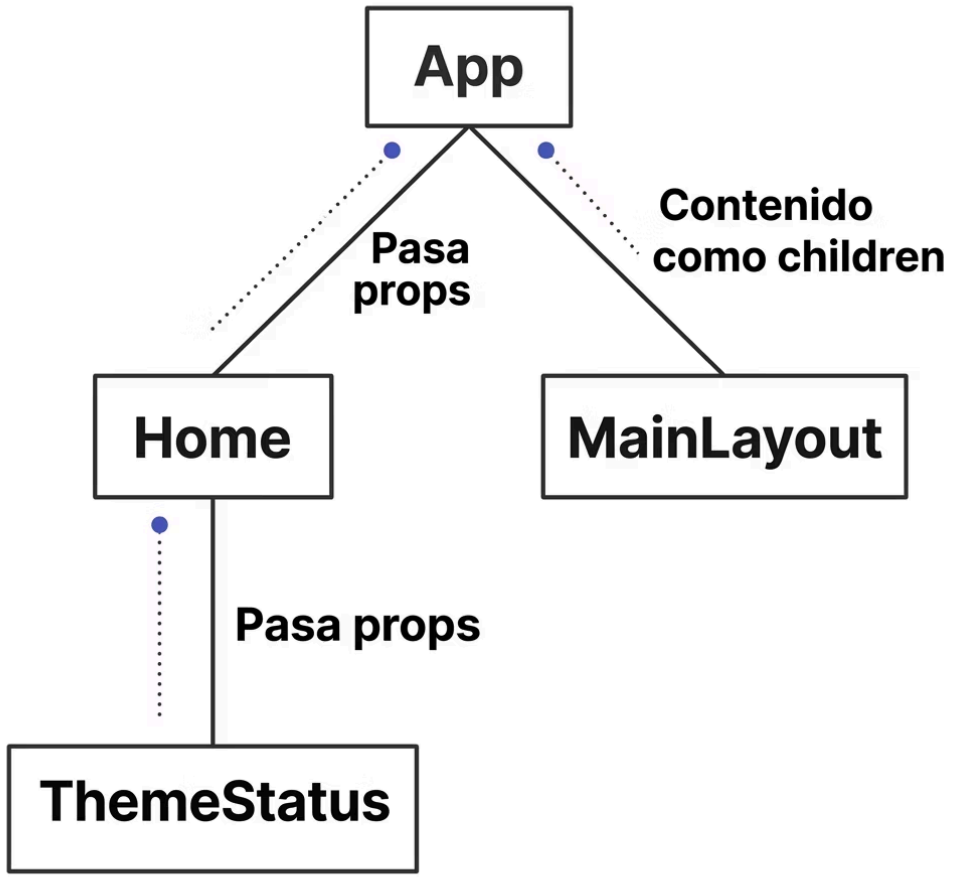
ThemeStatus.jsx

Alterna entre modo claro y oscuro, guardando la preferencia del usuario.

VisibilityToggle.jsx

Controla la visibilidad de elementos de la interfaz con un simple clic.

Props y Children en React

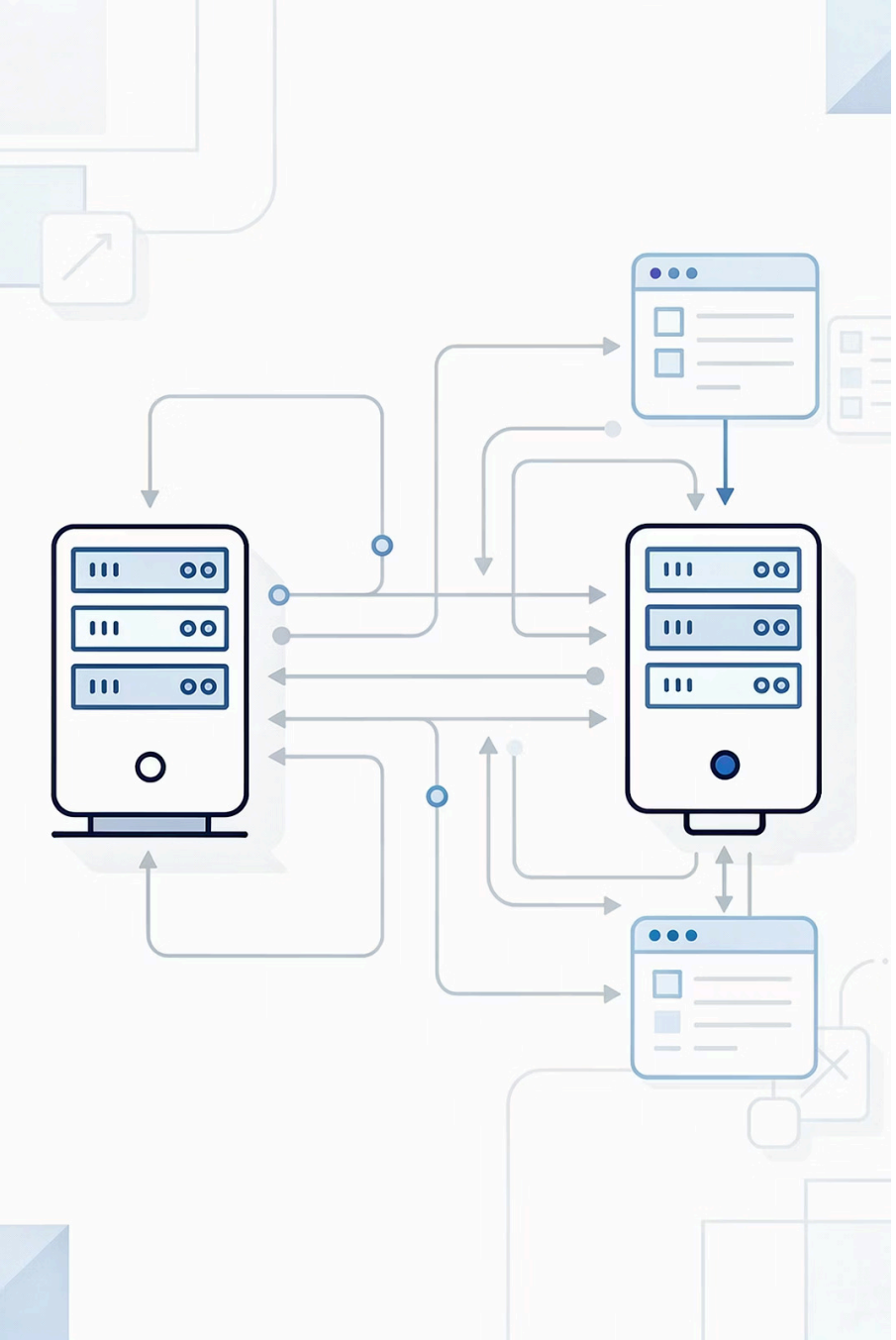


Props

Permiten enviar datos desde un componente **padre** hacia un componente **hijo**, configurando su comportamiento de forma dinámica.

Children

Permiten que componentes como **MainLayout** reciban contenido interno variable, funcionando como plantillas flexibles de la interfaz.



Conexión con la API en ASP.NET Core

El backend expone un endpoint REST a través del **ProductsController**, el cual devuelve datos en formato **JSON**. El frontend consume estos datos desde el archivo `services/api.js` utilizando **fetch**.

1

ASP.NET Core

ProductsController genera la respuesta JSON

2

HTTP Request

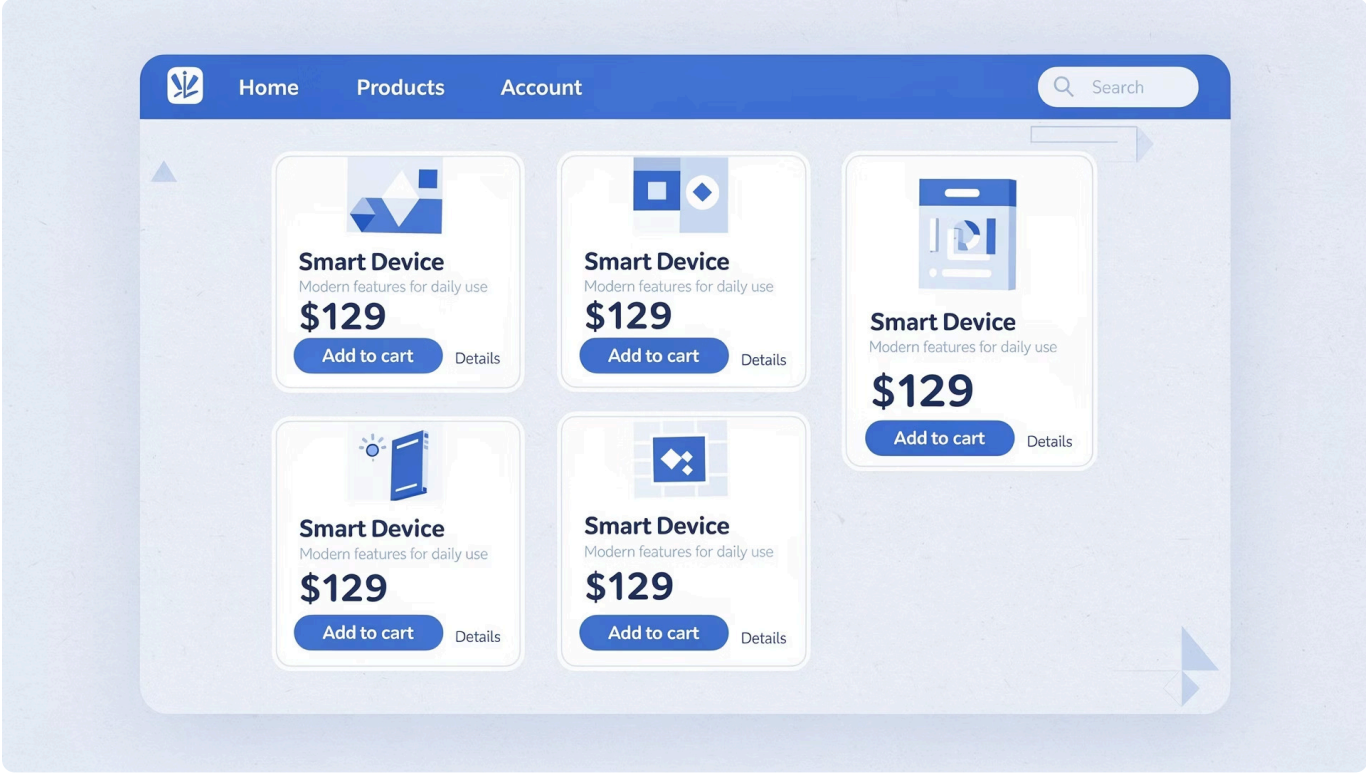
fetch() desde services/api.js

3

React

Datos recibidos y renderizados en la UI

Visualización de Datos desde la API



La página **Products** solicita los datos al backend al cargarse y los renderiza dinámicamente utilizando el componente **ProductCard**.

Obtención de datos

useEffect ejecuta fetch al montar el componente

Renderizado dinámico

Cada objeto JSON se mapea a un ProductCard

Integración real

Demuestra la comunicación completa entre React y ASP.NET Core

Conclusión

Este proyecto permitió aplicar de manera integral los conceptos fundamentales de la asignatura **Programación Web II**, construyendo una aplicación web moderna y funcional.

Componentes JSX

Arquitectura modular y reutilizable

Props y Children

Comunicación entre componentes

Estado React

Interfaces dinámicas con useState

Navegación

SPA con React Router

Consumo de API

Integración React + ASP.NET Core

¡Gracias por su atención!

Proyecto desarrollado para la asignatura **Programación Web II** · Docente: Johanna Rodriguez R.

Estudiante

Felix Hernandez

Matrícula

20-0143